

Aim:

You are an engineer aboard a cutting-edge ISRO interplanetary spacecraft en route to Mars. The navigation computer must perform lightning-fast 4×4 matrix multiplications to continuously adjust the spacecraft's flight trajectory.

To conserve fuel and reduce computational load, the system uses Strassen's Divide and Conquer Matrix Multiplication algorithm. Instead of multiplying directly, the 4×4 matrices are split into 2×2 quadrants and Strassen's method is applied recursively.

Algorithm Design (Strassen's for 4×4)

1. Divide Step

Split each 4×4 matrix into four 2×2 submatrices:

$A = \begin{bmatrix} A_{11} & A_{12} & A_{21} & A_{22} \end{bmatrix}, B = \begin{bmatrix} B_{11} & B_{12} & B_{21} & B_{22} \end{bmatrix}$

Each A_{ij}, B_{ij} is a 2×2 block.

2. Recursive Strassen Multiplication

Apply Strassen's 2×2 formulas to compute seven products:

- $P_1 = (A_{11} + A_{22})(B_{11} + B_{22})$
- $P_2 = (A_{21} + A_{22})B_{11}$
- $P_3 = A_{11}(B_{12} - B_{22})$
- $P_4 = A_{22}(B_{21} - B_{11})$
- $P_5 = (A_{11} + A_{12})B_{22}$
- $P_6 = (A_{21} - A_{11})(B_{11} + B_{12})$
- $P_7 = (A_{12} - A_{22})(B_{21} + B_{22})$

3. Combine Step

Construct the result matrix C from four 2×2 blocks:

- $C_{11} = P_1 + P_4 - P_5 + P_7$
- $C_{12} = P_3 + P_5$
- $C_{21} = P_2 + P_4$
- $C_{22} = P_1 - P_2 + P_3 + P_6$

4. Merge

- Place $C_{11}, C_{12}, C_{21}, C_{22}$ back into a 4×4 matrix.

Algorithm Analysis

Aspect	Classical Multiplication	Strassen's Multiplication
Multiplications for 2×2	8	7
Additions/Subtractions for 2×2	4	18
Multiplications for 4×4 (recursive)	64 (direct)	7 recursive calls on 2×2 blocks
Time Complexity	$O(n^3)$	$O(n \log_2^7 n) \cong O(n^{2.81})$
Space Complexity	$O(n^2)$	$O(n^2)$ (extra temporary matrices for recursion)
Efficiency	Simple, predictable	Faster for large matrices, especially $n > 128$

Input Format:

- The first 4 lines contain 4 space-separated integers each, representing the elements of the first 4×4 matrix A .
- The next 4 lines contain 4 space-separated integers each, representing the elements of the second 4×4 matrix B .

Output Format:

- Print the resulting 4×4 product matrix obtained using Strassen's matrix multiplication.

- Each row of the matrix should be printed on a new line with elements separated by spaces.

Note: Partial code is provided. Complete the missing logic in **add()**, **subtract()**, **strassen2()**, and **strassen4()** to implement Strassen's matrix multiplication.

Source Code:

stressensMatrixMultiplication.c

```
#include <stdio.h>

void add(int n, int A[n][n], int B[n][n], int C[n][n]) {
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            C[i][j]=A[i][j]+B[i][j];
    // Implement matrix addition
}

void subtract(int n, int A[n][n], int B[n][n], int C[n][n]) {
    // Implement matrix subtraction
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            C[i][j]=A[i][j]-B[i][j];
}

void strassen2(int A[2][2], int B[2][2], int C[2][2]) {
    int M1,M2,M3,M4,M5,M6,M7;
    M1=(A[0][0]+A[1][1])*(B[0][0]+B[1][1]);
    M2=(A[1][0]+A[1][1])*B[0][0];
    M3=A[0][0]*(B[0][1]-B[1][1]);
    M4=A[1][1]*(B[1][0]-B[0][0]);
    M5=(A[0][0]+A[0][1])*B[1][1];
    M6=(A[1][0]-A[0][0])*(B[0][0]+B[0][1]);
    M7=(A[0][1]-A[1][1])*(B[1][0]+B[1][1]);

    C[0][0] = M1 + M4 - M5 + M7;
    C[0][1] = M3 + M5;
    C[1][0] = M2 + M4;
    C[1][1] = M1 - M2 + M3 + M6;
}

void strassen4(int A[4][4], int B[4][4], int C[4][4]) {

    int A11[2][2],A12[2][2],A21[2][2],A22[2][2];
    int B11[2][2],B12[2][2],B21[2][2],B22[2][2];
    // Split the input matrices into 2 x 2 submatrices
    int P1[2][2], P2[2][2], P3[2][2], P4[2][2];
    int P5[2][2], P6[2][2], P7[2][2];
    int tempA[2][2], tempB[2][2];
    int i,j;
    for(i=0;i<2;i++){
        for(j=0;j<2;j++){
```

```

        A11[i][j]=A[i][j];
        A12[i][j]=A[i][j+2];
        A21[i][j]=A[i+2][j];
        A22[i][j]=A[i+2][j+2];

        B11[i][j]=B[i][j];
        B12[i][j]=B[i][j+2];
        B21[i][j]=B[i+2][j];
        B22[i][j]=B[i+2][j+2];
    }
}

// P1 = (A11 + A22)(B11 + B22)
add(2,A11,A22,tempA);
add(2,B11,B22,tempB);
strassen2(tempA,tempB,P1);

// P2 = (A21 + A22)B11
add(2,A21,A22,tempA);
strassen2(tempA,B11,P2);

// P3 = A11(B12 - B22)
subtract(2,B12,B22,tempB);
strassen2(A11,tempB,P3);

// P4 = A22(B21 - B11)
subtract(2,B21,B11,tempB);
strassen2(A22,tempB,P4);

// P5 = (A11 + A12)B22
add(2,A11,A12,tempA);
strassen2(tempA,B22,P5);

// P6 = (A21 - A11)(B11 + B12)
subtract(2,A21,A11,tempA);
add(2,B11,B12,tempB);
strassen2(tempA,tempB,P6);

// P7 = (A12 - A22)(B21 + B22)
subtract(2,A12,A22,tempA);
add(2,B21,B22,tempB);
strassen2(tempA,tempB,P7);

// Compute C submatrices
int C11[2][2],C12[2][2],C21[2][2],C22[2][2];
for(i=0;i<2;i++){
    for(j=0;j<2;j++){
        C11[i][j]=P1[i][j]+P4[i][j]-P5[i][j]+P7[i][j];
        C12[i][j]=P3[i][j]+P5[i][j];
        C21[i][j]=P2[i][j]+P4[i][j];
        C22[i][j]=P1[i][j]-P2[i][j]+P3[i][j]+P6[i][j];
    }
}

// Combine submatrices into C
for(i=0;i<2;i++){
    for(j=0;j<2;j++){

```

```

        C[i][j]=C11[i][j];
        C[i][j+2]=C12[i][j];
        C[i+2][j]=C21[i][j];
        C[i+2][j+2]=C22[i][j];
    }
}

int main() {
    int A[4][4], B[4][4], C[4][4];
    int i,j;
    printf("matrix A:\n");
    for (i = 0; i < 4; i++)
        for (j = 0; j < 4; j++)
            scanf("%d", &A[i][j]);
    printf("matrix B:\n");
    for ( i = 0; i < 4; i++)
        for (j = 0; j < 4; j++)
            scanf("%d", &B[i][j]);
    strassen4(A, B, C);
    printf("Product Matrix:\n");
    for ( i = 0; i < 4; i++) {
        for ( j = 0; j < 4; j++) {
            printf("%d ", C[i][j]);
        }
        printf("\n");
    }
    return 0;
}

```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
matrix A:
1 0 0 0
0 1 0 0
0 0 1 0
0 0 0 1
matrix B:
2 3 4 5
6 7 8 9
10 11 12 13
14 15 16 17
Product Matrix:
2 3 4 5
6 7 8 9
10 11 12 13
14 15 16 17

Test Case - 2
User Output
matrix A:
1 2 3 4
2 1 4 3
3 4 1 2
4 3 2 1
matrix B:
1 1 1 1
1 1 1 1
1 1 1 1
1 1 1 1
Product Matrix:
10 10 10 10
10 10 10 10
10 10 10 10
10 10 10 10